Experiential Learning Assignment-2
LLMOps: API Cost, Latency, and Prompt Version Management

Course: MLOps **Marks:** 10
Mode: Individual hands-on activity **Estimated Time:** 3–4 hours

Submission: Jupyter Notebook (.ipynb), exported PDF/HTML, CSV outputs, MLflow screenshots, and a short report (500–700 words)

1. Objective and Learning Outcomes

This assignment helps students operationalize an LLM application by measuring latency, calculating token-based costs, and managing prompt versions in MLflow. LLMOps extends MLOps by treating prompts, retrieval context, guardrails, and output quality as versioned production artifacts, while also managing inference cost, latency, and token consumption.developers.openai

After completing this assignment, students will be able to:

- Explain LLMOps and its differences from MLOps.
- Call an LLM through an API and measure response latency.
- Extract token usage and compute request-level and projected monthly costs.
- Register, compare, evaluate, and promote prompt versions in MLflow.
- Create a JSON-output prompt and validate it using a valid_json metric.

2. Background

MLOps focuses on managing datasets, feature pipelines, model training, deployment, and monitoring. **LLMOps** manages the full LLM application lifecycle, where prompts, model/API versions, retrieved context, and output guardrails can all change independently and affect quality.developers.openai

Key LLM-specific challenges include:

- **Cost:** API charges are commonly based on input and output token rates, so longer prompts and responses increase cost.
- **Latency:** Response time depends on prompt/context length, generated output length, model choice, and service/network conditions.developers.openai
- **Tokens:** Tokens determine both billing and context-window usage; API responses generally provide input, output, and total token counts.
- **Prompt drift:** Informal prompt edits can make outputs inconsistent and irreproducible, so prompts should be versioned and evaluated.
- **Non-determinism:** LLM outputs can vary, requiring automated checks such as JSON validity, safety checks, and human review.

3. Scenario

You are an LLMOps engineer at **EduAssist AI**, which is building an academic question-answering assistant. The team wants a prompt that returns strict JSON in this format:

```
json
{
  "answer": "string",
  "confidence": "high | medium | low",
  "key_points": ["string"]
}
```

You must test three prompt versions, measure their operational performance, and promote the best version to production in MLflow.

4. Tasks and Deliverables

Task 1: Call an LLM and Measure Latency — 2 Marks

Call an LLM API for **five academic questions** and measure latency for each request.

Requirements:

- Load the API key from an environment variable; do not hard-code it.
- Record question ID, prompt version, model name, start time, end time, latency, and response status.
- Calculate mean, median, minimum, maximum, and standard deviation of latency.
- Briefly explain the slowest request and possible reasons.

Deliverables:

- Executable API-call code
- Latency table for five requests
- Latency summary statistics
- Short interpretation paragraph

Task 2: Count Tokens and Calculate Cost — 3 Marks

Extract token usage from each API response and calculate cost using the selected model's published pricing. LLM APIs commonly bill separately for input and output tokens, and usage details are available in API responses.

Use:

Input Cost = Input Tokens/ 1,000,000 × Input Price per Million Tokens

Output Cost = Output Tokens / 1,000,000 ×Output Price per Million Tokens

Total Cost per Request = Input Cost + Output Cost

Projected Monthly Cost = Average Cost per Request × 10,000

Deliverables:

- Table showing input tokens, output tokens, total tokens, and cost per request
- Monthly cost projection for 10,000 requests
- One cost-optimization recommendation, such as reducing context size, shortening responses, caching stable prompt content, or using a smaller model for simple queries.

Task 3: Prompt Versioning and MLflow Evaluation — 5 Marks

Create an MLflow experiment named:

text

eduassist_llmops_assignment

Register and evaluate the following three prompts on the same five questions.

Prompt v1 — Basic

text

Answer the following student question clearly.

Question: {{question}}

Prompt v2 — Structured

text

You are an academic assistant.

Answer the following student question in a clear, concise, and accurate manner.

Use simple language suitable for undergraduate students.

Question: {{question}}

Prompt v3 — JSON Output

text

You are an academic assistant.

Answer the student question and return ONLY valid JSON.

Do not include markdown, explanations outside JSON, or code fences.

Use this exact JSON schema:

```
{
 "answer": "string",
 "confidence": "high | medium | low",
 "key_points": ["string"]
}
```

Question: {{question}}

MLflow's Prompt Registry supports centralized prompt storage, versioning, commit messages, diffs, evaluation, and environment aliases such as production.

Requirements:

- Register v1, v2, and v3 with meaningful commit messages.
- Run each prompt version on the same five questions.
- Log these metrics for each version:
    - avg_latency_seconds
    - avg_input_tokens
    - avg_output_tokens
    - avg_cost_per_request
    - valid_json
- For v3, calculate:

valid_json=Valid JSON ResponsesTotal Responses\text{valid_json} = \frac{\text{Valid JSON Responses}}{\text{Total Responses}}valid_json=Total ResponsesValid JSON Responses

MLflow can aggregate boolean feedback into an average, which serves as the JSON-validity pass rate.

- Compare all versions and promote the best one using the MLflow alias:

text

production

Deliverables:

- MLflow experiment screenshot showing runs and metrics
- Prompt Registry screenshot showing v1, v2, and v3
- Prompt diff or comparison evidence
- Prompt comparison table:

Prompt Version	Avg Latency	Avg Input Tokens	Avg Output Tokens	Avg Cost	Valid JSON Rate	Recommended?
v1						
v2						
v3						

- Production alias screenshot
- Promotion justification of at least 150 words, covering quality, JSON validity, latency, tokens, and cost

5. Marks Distribution

Component	Marks
Task 1: API call and latency measurement	2
Task 2: Token counting and cost economics	3
Task 3: MLflow prompt versioning, evaluation, and promotion	5
Total	**10**

6. Submission Checklist

Submit one ZIP file containing:

- Jupyter Notebook with executable code
- requirements.txt
- latency_tokens_cost.csv
- prompt_evaluation_results.csv
- MLflow screenshots: experiment, Prompt Registry, metrics, prompt diff, and production alias
- Final report in PDF format
- README with setup and reproduction steps

The README must include Python version, required libraries, environment variables, installation commands, MLflow UI command, and steps to reproduce the experiment.

7. Academic Integrity

- Never submit or expose API keys, billing information, or personal data.
- Use environment variables for secrets.
- Mention the model name, API version, pricing date, and currency.
- If API access is unavailable, use a locally hosted open-source model and clearly document the alternative; estimate token counts and costs using the model tokenizer and published hosting costs.